

Reviewer Pack — Invariant Ring Dimension and Circuit Complexity for Permanen...

Entry 56f24c51fad0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 23:23:42 UTC

Invariant Ring Dimension and Circuit Complexity for Permanent vs Determinant

Entry ID: 56f24c51fad0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 23:23:42 UTC

1. Conjecture

Field A (mathematical branch): Invariant Theory **Field B** (complexity object): Circuit Complexity

Statement:

Let P_n be the permanent polynomial and D_n the determinant polynomial on $n \times n$ matrices. For each $n \geq 2$, the dimension of the invariant ring of $GL(n)$ acting on the space of $n \times n$ matrices is exponentially larger for P_n than for D_n , implying that the minimal circuit size for P_n is exponentially greater than that for D_n .

Rationale (proposer's reasoning):

Invariant theory under $GL(n)$ provides tools to distinguish algebraic varieties like the permanent and determinant. The dimension of the invariant ring reflects the complexity of the orbit closure, which is hypothesized to correlate with circuit complexity. This conjecture leverages GCT's focus on orbit closures and representation-theoretic multiplicities to separate VP and VNP.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c82d68af5d9034e6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def matrix_multiply(A, B):
        n = len(A)
        C = [[0 for _ in range(n)] for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            for j in range(i+1, n):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
                b[j] -= factor * b[i]
        x = [0 for _ in range(n)]
        for i in range(n-1, -1, -1):
            x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
        return x

    def determinant(A):
        if len(A) == 2:
            return A[0][0]*A[1][1] - A[0][1]*A[1][0]
        det = 0
```

```

    for j in range(len(A)):
        det += ((-1)**j) * A[0][j] * determinant([row[:j] + row[j+1:] for row in A[1:]])
    return det

def permanent(A):
    if len(A) == 2:
        return A[0][0]*A[1][1] + A[0][1]*A[1][0]
    perm = 0
    for j in range(len(A)):
        perm += ((-1)**j) * A[0][j] * permanent([row[:j] + row[j+1:] for row in A[1:]])
    return perm

def invariant_ring_dimension(n):
    if n == 2:
        return 3
    elif n == 3:
        return 6
    else:
        return "mapping_undefined"

n = random.randint(5, 40)
P_n = permanent([[random.randint(-10, 10) for _ in range(n)] for _ in range(n)])
D_n = determinant([[random.randint(-10, 10) for _ in range(n)] for _ in range(n)])

dim_P_n = invariant_ring_dimension(n)
dim_D_n = invariant_ring_dimension(n)

if dim_P_n == "mapping_undefined" or dim_D_n == "mapping_undefined":
    return {
        "metric_name": "invariant_ring_dimension",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

return {
    "metric_name": "invariant_ring_dimension",
    "metric_value": dim_P_n - dim_D_n,
    "instances_tested": 1,
    "conjecture_holds": dim_P_n > dim_D_n,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30)) + list(range(30, 40))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_value"] is not None) / len(results))

```

```

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"invariant_ring_dimension\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing definitive validation or falsification. |
next: Increase timeout duration and re-run test with extended computation limits

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136566
2	propose	ollama_remote	qwen3:8b	0	0	189547
3	preregistration	ollama_remote	qwen3:8b	0	0	31712
4	novelty	ollama_remote	qwen3:8b	0	0	30000
5	novelty	ollama_remote	qwen3:8b	0	0	23495
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37591
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12853
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11172
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12629
10	judge	ollama_remote	qwen3:8b	0	0	20884

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 506449 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/56f24c51fad0.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/56f24c51fad0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/56f24c51fad0.tar.gz` (if generated)